

ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ

Εργαστήριο Ηλεκτρονικών Αισθητηρίων

ΜΑΘΗΜΑ: Τεχνολογία Αισθητήρων και Μικροσυστημάτων

3η και 4η Εργαστηριακή Άσκηση

Arduino και εφαρμογές

Ονοματεπώνυμο: Δημήτριος Μιχαλακόπουλος (03120229)

Κωνσταντίνος Μπάρκας (03120849)

Ομάδα: 1

Εξάμηνο: 10ο

Ημερομηνία διεξαγωγής: Πέμπτη 20 Μαρτίου 2025

Πέμπτη 3 Απριλίου 2025

Παράδειγμα 1: Blink

Ζητούμενο 1

Τροποποιήστε τον κώδικα ώστε το LED να μένει αναμμένο για 1.5 s και σβηστό για 0.5 s.

Κώδικας

```
void setup() {
  pinMode(LED_BUILTIN, OUTPUT);  // Initialize the built-in LED pin as output
}

void loop() {
  digitalWrite(LED_BUILTIN, HIGH);  // Turn the LED ON
  delay(1500);                      // Keep it ON for 1.5 seconds (1500 ms)
  digitalWrite(LED_BUILTIN, LOW);   // Turn the LED OFF
  delay(500);                       // Keep it OFF for 0.5 seconds (500 ms)
}
```

Επεξήγηση

Αρχικά, στην setup() αρχικοποιούμε το ενσωματωμένο LED ως έξοδο. Έπειτα, στην loop() ανάβουμε το LED (το θέτουμε σε HIGH), το κρατάμε αναμμένο για 1.5 δευτερόλεπτα και μετά το σβήνουμε (το θέτουμε σε LOW) και το κρατάμε σβηστό για 0.5 δευτερόλεπτα.

Ζητούμενο 2

Τροποποιήστε τον κώδικα ώστε το LED να εκπέμπει SOS σε σήματα Morse βάσει των παρακάτω:

S	...
O	---
Παύση μεταξύ των συμβόλων (. ή -)	.
Παύση μεταξύ των γραμμάτων	-

όπου η **τελεία (.)** αντιστοιχεί σε **0.5 s** και η **παύλα (-)** σε **1.5 s**.

Κώδικας

```
void setup() {
  pinMode(LED_BUILTIN, OUTPUT);  // Set LED pin as output
}

void loop() {
  morseS();  // Send S (···)
  morseO();  // Send O (---)
  morseS();  // Send S (···)
  delay(3000); // Pause before repeating SOS
}

// Function for Morse "S" (···)
void morseS() {
  for (int i = 0; i < 3; i++) {
    digitalWrite(LED_BUILTIN, HIGH); // Dot = 0.5s ON
    delay(500);
    digitalWrite(LED_BUILTIN, LOW);  // Pause between dots = 0.5s OFF
    delay(500);
  }
  delay(1000); // Add 1s more for total pause between letters = 1.5s
}

// Function for Morse "O" (---)
void morseO() {
  for (int i = 0; i < 3; i++) {
    digitalWrite(LED_BUILTIN, HIGH); // Dash = 1.5s ON
    delay(1500);
    digitalWrite(LED_BUILTIN, LOW);  // Pause between dashes = 0.5s OFF
    delay(500);
  }
  delay(1000); // Add 1s more for total pause between letters = 1.5s
}
```

Επεξήγηση

Αρχικά, στην setup() αρχικοποιούμε το ενσωματωμένο LED ως έξοδο. Έπειτα, στην loop() καλούνται διαδοχικά οι συναρτήσεις morseS() και morseO() για να αποδοθεί το "S" (· · ·) και το "O" (- - -). Η morseS() αναβοσβήνει το LED τρεις φορές για 0,5 sec με 0,5 sec παύση, αποδίδοντας τις τελείες. Η morseO() κρατά το LED αναμμένο για 1,5 sec τρεις φορές με ενδιάμεσες παύσεις 0,5 sec, αποδίδοντας τις παύλες. Μετά από κάθε γράμμα, προστίθεται παύση 1 sec για συνολικό διάστημα 1,5 sec μεταξύ των γραμμάτων. Τέλος, βάζουμε μία παύση 3 sec πριν επαναληφθεί το μήνυμα " SOS ".

Παράδειγμα 2: Έλεγχος LED μέσω σειριακής επικοινωνίας

Ζητούμενο

Τροποποιήστε τον κώδικα ώστε:

- Το LED να ενεργοποιείται με τους χαρακτήρες H ή h και να απενεργοποιείται με τους χαρακτήρες L ή l.
- Να εμφανίζεται ένα μήνυμα σφάλματος εάν πληκτρολογηθεί οποιοσδήποτε άλλος χαρακτήρας.

Κώδικας

```
const int ledPin = 13;  // Pin number where the LED is connected
int incomingByte;       // Variable to store incoming serial data

void setup() {
  Serial.begin(9600);           // Start serial communication at 9600 baud
  pinMode(ledPin, OUTPUT);      // Set the LED pin as an output
}

void loop() {
  // Check if any data is available to read from the serial buffer
  if (Serial.available() > 0) {
    incomingByte = Serial.read();  // Read the incoming byte

    // Turn the LED on if the input is 'H' or 'h'
    if (incomingByte == 'H' || incomingByte == 'h') {
      digitalWrite(ledPin, HIGH);
    }
    // Turn the LED off if the input is 'L' or 'l'
    else if (incomingByte == 'L' || incomingByte == 'l') {
      digitalWrite(ledPin, LOW);
    }
    // If any other character is received, print an error message
    else {
      Serial.println("Error!");
    }
  }
}
```

Επεξήγηση

Αρχικά, ορίζουμε την σταθερά ledPin που αντιστοιχεί στο ενσωματωμένο LED (pin D13) και την μεταβλητή incomingByte, στην οποία θα αποθηκεύεται το byte που θα στέλνουμε μέσω σειριακής επικοινωνίας. Έπειτα, στην setup()

ξεκινάει η σειριακή επικοινωνία με ταχύτητα 9600 baud και ορίζουμε το pin του LED ως έξοδο. Στην συνέχεια, στην loop() το πρόγραμμα ελέγχει συνεχώς αν υπάρχουν δεδομένα διαθέσιμα στη σειριακή θύρα. Έτσι, όταν ληφθεί ένας χαρακτήρας, διαβάζεται και ελέγχεται: αν είναι 'H' ή 'h', το LED ανάβει, αλλιώς αν είναι 'L' ή 'l', το LED σβήνει. Σε οποιαδήποτε άλλη περίπτωση εμφανίζεται το μήνυμα "Error!" στη σειριακή οθόνη.

Παράδειγμα 3: Αισθητήρας θερμοκρασίας TMP36

Ζητούμενο 1

Τροποποιήστε τον κώδικα ώστε να ανάβουν διαδοχικά όλα τα LEDs αναλόγως της θερμοκρασίας, δηλαδή:

Θερμοκρασία	LEDs
$\theta \leq 25$	Πράσινο
$25 < \theta \leq 35$	Πράσινο & Πορτοκαλί
$\theta > 35$	Πράσινο & Πορτοκαλί & Κόκκινο

Κώδικας

```
int green_pin = 2;     // Pin for the green LED
int orange_pin = 3;    // Pin for the orange LED
int red_pin = 4;       // Pin for the red LED
int tempPin = A0;      // Analog pin connected to the temperature sensor

void setup() {
  pinMode(green_pin, OUTPUT);     // Set green LED pin as output
  pinMode(orange_pin, OUTPUT);    // Set orange LED pin as output
  pinMode(red_pin, OUTPUT);       // Set red LED pin as output
  Serial.begin(9600);             // Initialize serial communication at 9600
baud rate
}

void loop() {
  int tempValue = analogRead(tempPin);  // Read the analog value from the
temperature sensor
  float temp = tempValue * 5000.0 / 1024.0;  // Convert analog value to
millivolts
  float temp_C = (temp - 500) / 10;          // Convert millivolts to degrees
Celsius
```

```
  Serial.println(tempValue);          // Print the raw sensor value
  Serial.print("Temp = ");
  Serial.print(temp_C);               // Print the calculated temperature
  Serial.println("C");

  // Turn on LEDs based on temperature range
  if (temp_C <= 25) {
    digitalWrite(green_pin, HIGH);    // Only green LED ON
    digitalWrite(orange_pin, LOW);
    digitalWrite(red_pin, LOW);
  }
  else if (temp_C > 25 && temp_C <= 35) {
    digitalWrite(green_pin, HIGH);    // Green and Orange LEDs ON
    digitalWrite(orange_pin, HIGH);
    digitalWrite(red_pin, LOW);
  }
  else {
    digitalWrite(green_pin, HIGH);    // All LEDs ON
    digitalWrite(orange_pin, HIGH);
    digitalWrite(red_pin, HIGH);
  }

  delay(1000); // Wait for 1 second before next reading
}
```

Επεξήγηση

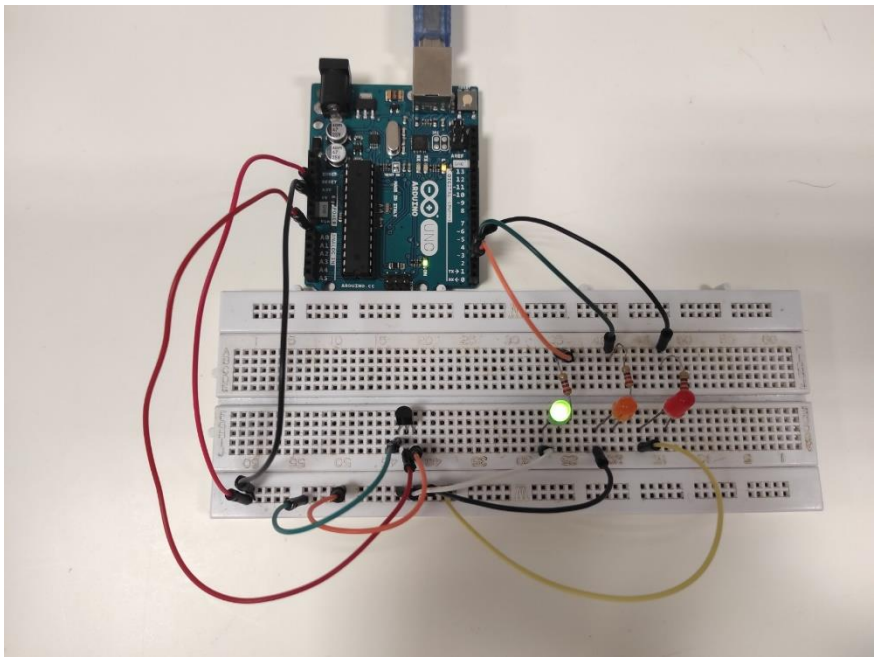
Αρχικά, ορίζουμε τρεις μεταβλητές που αποθηκεύουν τους αριθμούς των pins που είναι συνδεδεμένα το πράσινο, το πορτοκαλί και το κόκκινο LED, καθώς και μία μεταβλητή που αποθηκεύει τον αριθμό του αναλογικού pin που είναι συνδεδεμένος ο αισθητήρας θερμοκρασίας. Έπειτα, στην setup() τα pins των LEDs ορίζονται ως έξοδοι και ξεκινά η σειριακή επικοινωνία στα 9600 baud. Στην συνέχεια, στην loop() το πρόγραμμα διαβάζει την αναλογική τιμή από τον αισθητήρα, τη μετατρέπει σε τάση και έπειτα σε θερμοκρασία σε βαθμούς Κελσίου. Η θερμοκρασία εκτυπώνεται στη σειριακή οθόνη. Αν η θερμοκρασία είναι έως 25°C, ανάβει μόνο το πράσινο LED. Αν είναι μεταξύ 25°C και 35°C, ανάβουν το πράσινο και το πορτοκαλί LED. Αν η θερμοκρασία ξεπεράσει τους 35°C, ανάβουν και τα τρία LEDs. Το πρόγραμμα λαμβάνει μια νέα μέτρηση από τον αισθητήρα θερμοκρασίας κάθε 1 sec.

Η μετατροπή από την τιμή του 10 bit ADC του Arduino σε τάση γίνεται ως εξής:

$$\text{Για } V_{cc} = 5\ V = 5000\ mV\text{:} \qquad V_{out} = \text{τιμή_εισόδου} \cdot \frac{5000}{1024}$$

Από το datasheet του αισθητήρα θερμοκρασίας TMP36 έχουμε ότι η σχέση τάσης εξόδου και θερμοκρασίας είναι 10 mV/ °C, με ένα offset 500 mV, άρα η μετατροπή από τάση σε θερμοκρασία γίνεται με τον εξής τύπο:

$$temp_C = \frac{V_{out} - 500}{10}$$



Ζητούμενο 2

Έστω ότι χρησιμοποιούμε τάση τροφοδοσίας 3.3 V και 12-bit ADC. Ποιες τροποποιήσεις πρέπει να γίνουν στον κώδικα;

Κώδικας

Ο κώδικας είναι ο ίδιος με το προηγούμενο ζητούμενο με μόνη αλλαγή στην γραμμή 15, όπου γίνεται η μετατροπή από την τιμή ADC σε τάση:

```
float temp = tempValue * 3300.0 / 4096.0;
```

Επεξήγηση

Η μετατροπή από 12 bit ADC σε τάση γίνεται με τον παρακάτω τύπο:

$$Για\ V_{cc} = 3.3\ V = 3300\ mV: \qquad V_{out} = τιμή_εισόδου \cdot \frac{3300}{4096}\ \ (2^{12} = 4096)$$

Η μετατροπή από τάση σε θερμοκρασία γίνεται με το ίδιο τύπο όπως και πριν:

$$temp_C = \frac{V_{out} - 500}{10}$$

Παράδειγμα 4: Hall SS49

Ζητούμενο 1

Τροποποιήστε τον κώδικα ώστε κατά την απουσία του μαγνήτη να εμφανίζεται η τιμή 0 mT.

Κώδικας

```
int pin = A0;          // Input pin for the Hall sensor
int vcc = 5000;        // Supply voltage for the Hall sensor in millivolts (mV)

int value;             // Variable to store the analog reading (0-1023)
int voltage;           // Variable to store the voltage value in mV
int field;             // Variable to store the magnetic field value in
millitesla (mT)

void setup() {
  Serial.begin(9600); // Start serial communication at 9600 baud
}

void loop() {
  value = analogRead(pin); // Read the analog value from the Hall sensor

  // Convert the analog value to voltage (in mV)
  voltage = value * (vcc / 1024.0);

  // Calculate the magnetic field strength in millitesla (mT)
  field = map(voltage, 1000, 4000, -100, 100);

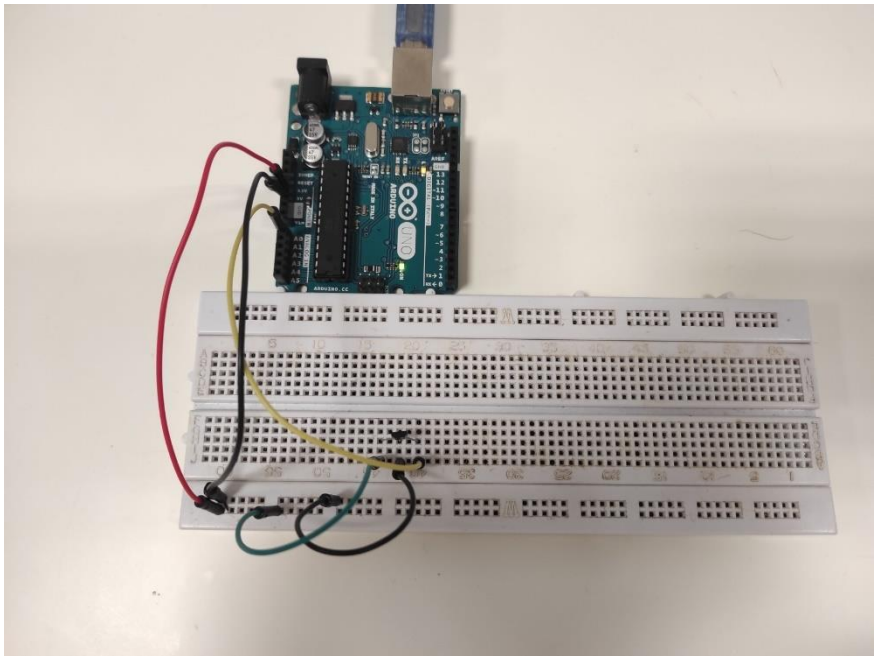
  field--; // Apply a correction by decreasing the field value by 1 mT

  // Print the sensor reading, voltage, and magnetic field to the Serial
Monitor
  Serial.print("Value: ");
  Serial.print(value);       // Print the raw analog value (0-1023)
  Serial.print("\t");
  Serial.print(voltage);     // Print the voltage in millivolts
  Serial.println(" mV");
  Serial.print("Magnetic Field: ");
  Serial.print(field);       // Print the calculated magnetic field value
  Serial.println(" mT");
  Serial.println("=========================="); // Separator line for
readability

  delay(1000); // Wait 1 second before taking the next reading
}
```

Επεξήγηση

Αρχικά, ορίζουμε μία μεταβλητή που αποθηκεύει τον αριθμό του αναλογικού pin που είναι συνδεδεμένος ο αισθητήρας Hall και μια μεταβλητή για την τάση τροφοδοσίας του. Έπειτα, στην setup() ξεκινά η σειριακή επικοινωνία στα 9600 baud. Στην συνέχεια, στην loop() το πρόγραμμα διαβάζει την αναλογική τιμή από τον αισθητήρα (value), τη μετατρέπει σε τάση σε millivolt (voltage), και έπειτα υπολογίζει την τιμή του μαγνητικού πεδίου σε millitesla (field) χρησιμοποιώντας τη συνάρτηση map() για να προσαρμόσει την κλίμακα από 1000 - 4000 mV σε -100 - 100 mT. Κατόπιν μειώνει το αποτέλεσμα κατά 1 για να καλιμπράρει τον αισθητήρα σε τιμή 0 mT κατά την απουσία μαγνήτη. Όλες οι τιμές εκτυπώνονται στη σειριακή οθόνη για παρακολούθηση. Ο κώδικας κάνει μετρήσεις και ενημερώνει τα αποτελέσματα κάθε ένα δευτερόλεπτο.



Ζητούμενο 2

Τροποποιήστε το κύκλωμα και τον παραπάνω κώδικα ώστε να ενεργοποιείται ένα LED όταν η τιμή του μαγνητικού πεδίου που ανιχνεύει ο αισθητήρας Hall ξεπερνά κατ’ απόλυτη τιμή την τιμή των 5 mT.

Κώδικας

```
int pin = A0;          // Input pin for the Hall sensor
int led = 3;           // Output pin for the LED
int vcc = 5000;        // Supply voltage for the Hall sensor in millivolts (mV)
```

```
int value;            // Variable to store the analog reading (0-1023)
int voltage;          // Variable to store the voltage value in mV
int field;            // Variable to store the magnetic field value in
millitesla (mT)

void setup() {
  pinMode(led, OUTPUT);    // Set the LED pin as an output
  Serial.begin(9600);      // Start serial communication at 9600 baud
}

void loop() {
  value = analogRead(pin); // Read the analog value from the Hall sensor

  // Convert the analog value to voltage (in mV)
  voltage = value * (vcc / 1024.0);

  // Calculate the magnetic field strength in millitesla (mT)
  field = map(voltage, 1000, 4000, -100, 100);

  field--; // Apply a correction by decreasing the field value by 1 mT

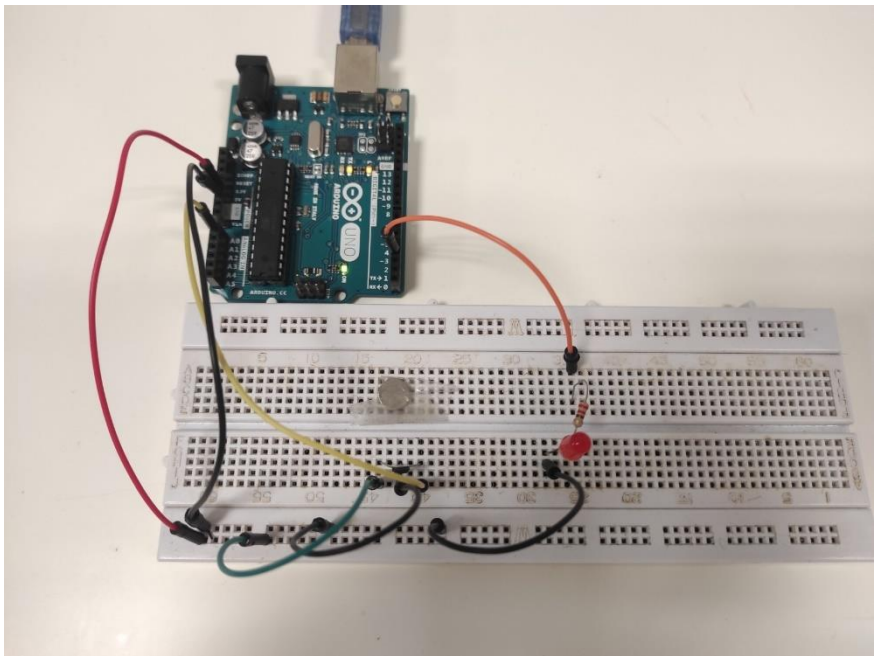
  // Check if the magnetic field is greater than +5 mT or less than -5 mT
  if (field > 5 || field < -5) {
    digitalWrite(led, HIGH); // Turn on the LED if the magnetic field is
strong
  }
  else {
    digitalWrite(led, LOW);  // Turn off the LED if the magnetic field is weak
  }

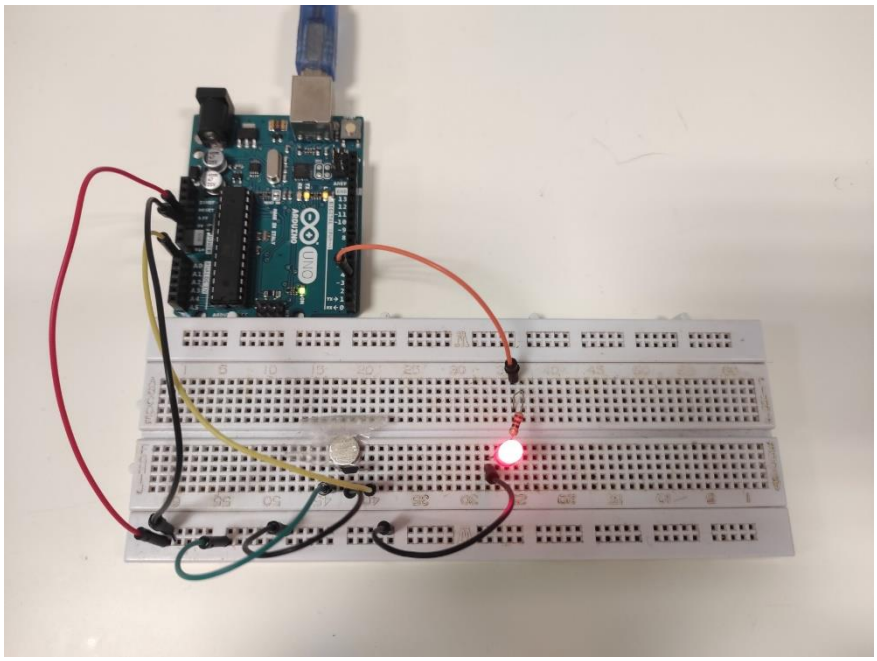
  // Print the sensor reading, voltage, and magnetic field to the Serial
Monitor
  Serial.print("Value: ");
  Serial.print(value);       // Print the raw analog value (0-1023)
  Serial.print("\t");
  Serial.print(voltage);     // Print the voltage in millivolts
  Serial.println(" mV");
  Serial.print("Magnetic Field: ");
  Serial.print(field);       // Print the calculated magnetic field value
  Serial.println(" mT");
  Serial.println("=========================="); // Separator line for
readability

  delay(1000); // Wait 1 second before taking the next reading
}
```

Επεξήγηση

Αρχικά, ορίζουμε μία μεταβλητή που αποθηκεύει τον αριθμό του ψηφιακού pin που είναι συνδεδεμένο το LED, μία μεταβλητή που αποθηκεύει τον αριθμό του αναλογικού pin που είναι συνδεδεμένος ο αισθητήρας Hall και άλλη μία μεταβλητή για την τάση τροφοδοσίας του. Έπειτα, στην setup() το LED pin ορίζεται ως έξοδος και ξεκινά η σειριακή επικοινωνία στα 9600 baud. Στην συνέχεια, στην loop() το πρόγραμμα λαμβάνει μετρήσεις όπως και παραπάνω. Επιπλέον, τώρα αν το μαγνητικό πεδίο είναι μεγαλύτερο κατά απόλυτη τιμή από 5 mT το LED ανάβει· διαφορετικά, το LED παραμένει σβηστό.





Ζητούμενο 3

Αλλάξτε τη διάρκεια παύσης μεταξύ των μετρήσεων σε 100 ms. Στη συνέχεια, τροποποιήστε τον κώδικα ώστε η φωτεινότητα του LED να είναι ανάλογη της απόλυτης τιμής του μαγνητικού πεδίου που ανιχνεύει ο αισθητήρας Hall.

Κώδικας

```
int pin = A0;          // Input pin for the Hall sensor
int led = 3;           // Output pin for the LED
int vcc = 5000;        // Supply voltage for the Hall sensor in millivolts (mV)

int value;             // Variable to store the analog reading (0-1023)
int voltage;           // Variable to store the voltage value in mV
int field;             // Variable to store the magnetic field value in
millitesla (mT)
int brightness;        // Variable to store the LED brightness level (0-255)

void setup() {
  pinMode(led, OUTPUT);    // Set the LED pin as an output
  Serial.begin(9600);      // Start serial communication at 9600 baud
}

void loop() {
  value = analogRead(pin); // Read the analog value from the Hall sensor

  // Convert the analog value to voltage (in mV)
  voltage = value * (vcc / 1024.0);

  // Calculate the magnetic field strength in millitesla (mT)
  field = map(voltage, 1000, 4000, -100, 100);

  field--; // Apply a correction by decreasing the field value by 1 mT

  // Map the absolute value of the magnetic field (0 to 100 mT) to a
brightness range (0 to 255)
  brightness = map(abs(field), 0, 100, 0, 255);

  // Set the LED brightness based on the magnetic field strength
  analogWrite(led, brightness);

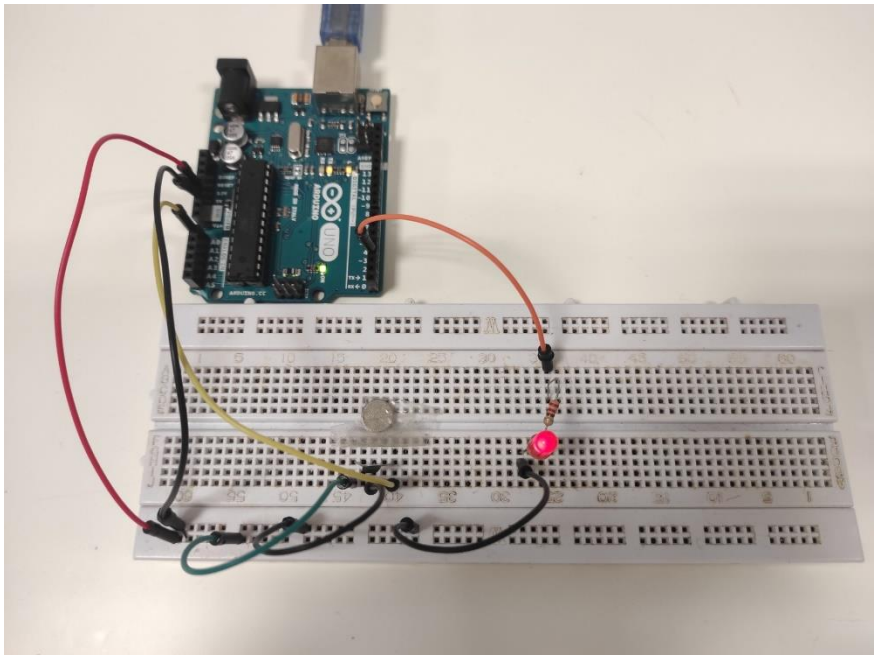
  // Print the sensor reading, voltage, and magnetic field to the Serial
Monitor
  Serial.print("Value: ");
  Serial.print(value);       // Print the raw analog value (0-1023)
  Serial.print("\t");
  Serial.print(voltage);     // Print the voltage in millivolts
  Serial.println(" mV");
```

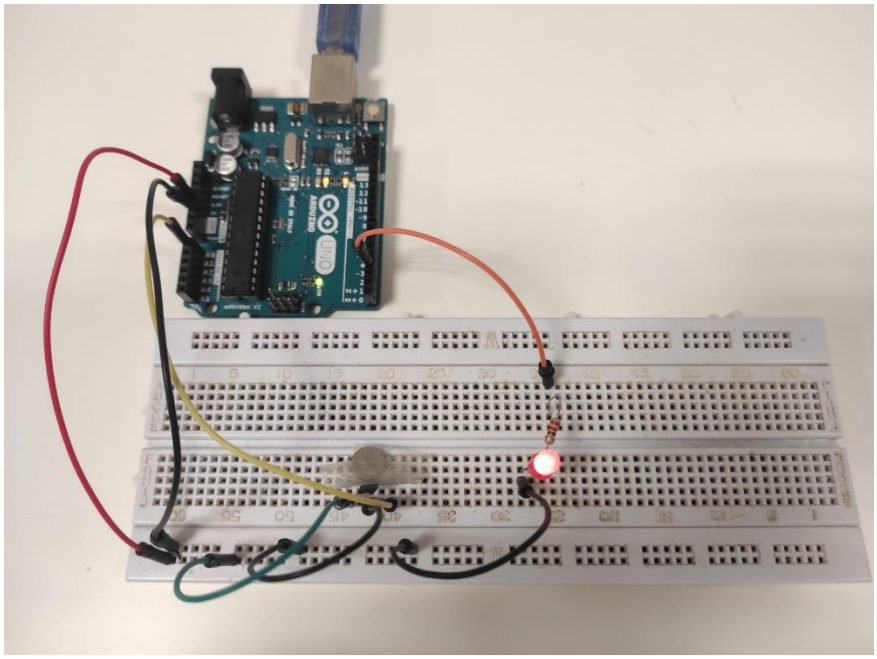
```
  Serial.print("Magnetic Field: ");
  Serial.print(field);       // Print the calculated magnetic field value
  Serial.println(" mT");
  Serial.println("=========================="); // Separator line for
readability

  delay(100); // Wait 100 milliseconds before taking the next reading
}
```

Επεξήγηση

Αρχικά, ορίζουμε μία μεταβλητή που αποθηκεύει τον αριθμό του ψηφιακού pin που είναι συνδεδεμένο το LED, μία μεταβλητή που αποθηκεύει τον αριθμό του αναλογικού pin που είναι συνδεδεμένος ο αισθητήρας Hall και άλλη μία μεταβλητή για την τάση τροφοδοσίας του. Έπειτα, στην setup() το LED pin ορίζεται ως έξοδος και ξεκινά η σειριακή επικοινωνία στα 9600 baud. Στην συνέχεια, στην loop() το πρόγραμμα λαμβάνει μετρήσεις όπως και παραπάνω. Επιπλέον, τώρα η φωτεινότητα του LED είναι ανάλογη της απόλυτης τιμής του μαγνητικού πεδίου, δηλαδή όσο πιο ισχυρό είναι το μαγνητικό πεδίο τόσο πιο μεγάλη θα είναι και η ένταση φωτός του LED.





Παράδειγμα 5: Αισθητήρας φωτεινότητας

Ζητούμενο 1

Εξηγήστε τον ρόλο του διαιρέτη τάσης στο κύκλωμα.

Επεξήγηση

Ο ρόλος του διαιρέτη τάσης στο κύκλωμα είναι να ανιχνεύει τις μεταβολές στην ένταση του φωτός, σύμφωνα με τον παρακάτω τύπο:

$$V_{out} = V_{CC} \frac{R_2}{R_1 + R_2}$$

Συγκεκριμένα, όταν η ένταση του φωτός αυξάνεται, η φωτοαντίσταση (R1) μειώνεται, με αποτέλεσμα η V_{out} να αυξάνεται. Αντίθετα, όταν η ένταση του φωτός μειώνεται, η φωτοαντίσταση (R1) αυξάνεται, με αποτέλεσμα η V_{out} να ελαττώνεται. Έτσι, το Arduino μπορεί άμεσα να ανιχνεύει τις συνθήκες φωτός της φωτοαντίστασης και να ελέγχει το LED, αφού η αναλογική τάση εισόδου που διαβάζει μετατρέπεται μέσω του ADC σε μία τιμή που μπορεί πολύ πιο εύκολα να χρησιμοποιηθεί στο πρόγραμμά μας.

Ζητούμενο 2

Αντικαταστήστε το κύκλωμα του LED με ένα ηχείο (buzzer). Πραγματοποιήστε τις απαραίτητες αλλαγές, ώστε η φωτοαντίσταση να μεταβάλλει, μέσω της συνάρτησης analog(), τη συχνότητα που αναπαράγεται από το buzzer, κατά το εύρος 100-1000 Hz. Γι' αυτό τον σκοπό, χρησιμοποιήστε τη συνάρτηση tone().

Κώδικας

```
int incomingByte;    // Variable to store incoming serial data
int value;           // Variable to store sensor reading

void setup() {
  pinMode(11, OUTPUT);        // Set pin 11 as an output (for buzzer or LED)
  Serial.begin(9600);         // Start serial communication at 9600 baud
  Serial.println("Please enter A for Analog or D for Digital:");  // Prompt
user for input
}

void loop() {
  // Check if data is available in the Serial buffer
  if (Serial.available() > 0) {
    incomingByte = Serial.read();   // Read the incoming byte
    if (incomingByte == 'A') {
      // If 'A' is received, keep running analog mode until new input
      while (Serial.available() == 0) {
        analog();
      }
    }
    else if (incomingByte == 'D') {
      // If 'D' is received, keep running digital mode until new input
      while (Serial.available() == 0) {
        digital();
      }
    }
    else {
      // If an unknown command is received, print an error message
      Serial.println("Unknown command");
    }
  }
}

void analog() {
  value = analogRead(A0);      // Read analog value from sensor (A0)
  Serial.print(value);         // Print raw analog value
  Serial.print("\t");
```

```
  int analogValue = map(value, 512, 1023, 100, 1000);  // Map value to
frequency range for tone

  tone(11, analogValue, 20);   // Generate a tone on pin 11 with the mapped
frequency for 20 ms
  Serial.println(analogValue); // Print mapped analog value (frequency)
}

void digital() {
  value = analogRead(A0);      // Read analog value from sensor (A0)
  Serial.print(value);         // Print raw analog value
  Serial.print("\t");

  boolean digitalValue = map(value, 0, 600, HIGH, LOW);  // Map value to
HIGH/LOW digital output

  digitalWrite(11, digitalValue); // Set pin 11 HIGH or LOW based on mapped
value
  Serial.println(digitalValue);   // Print digital value (1 or 0)
}
```

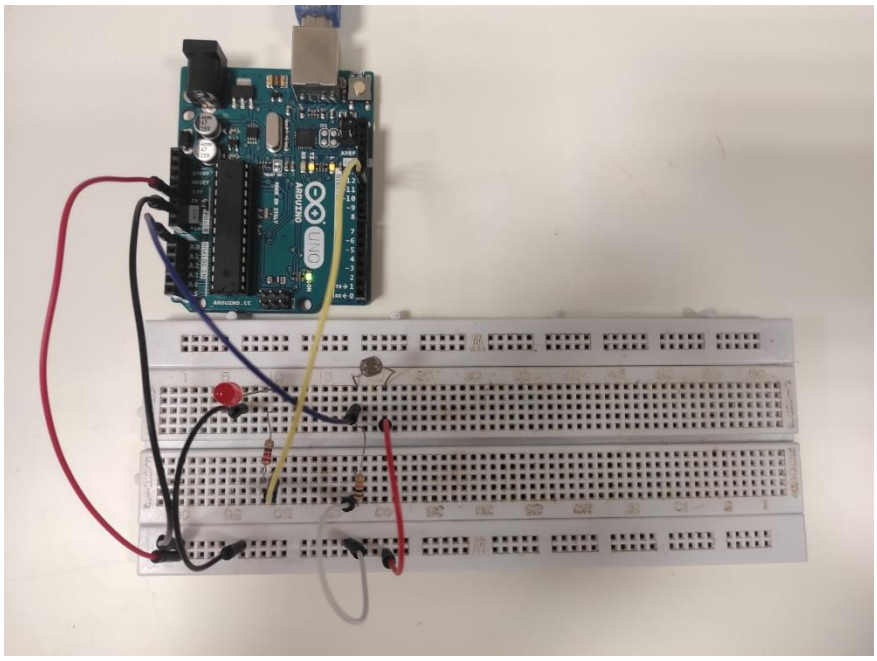
Επεξήγηση

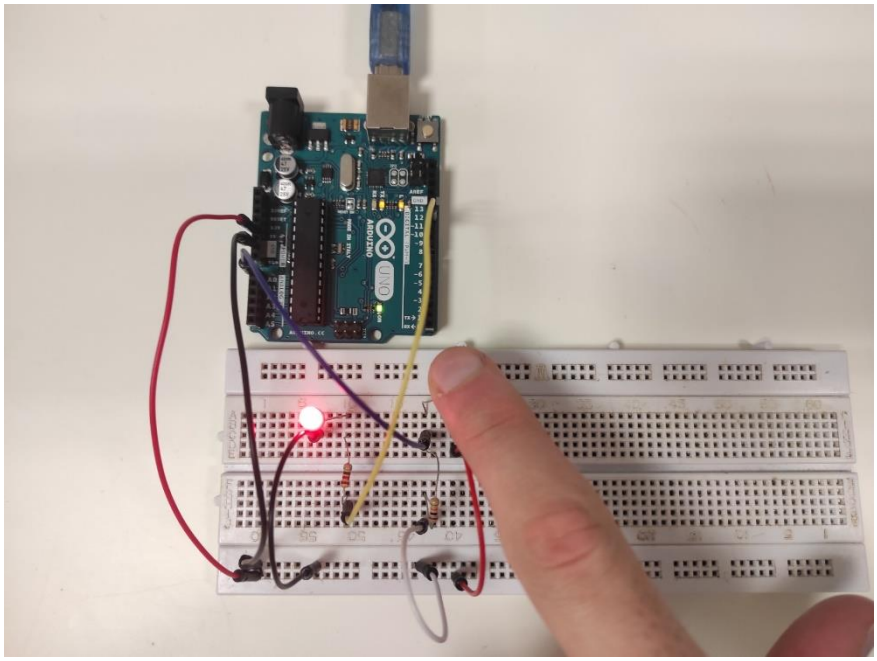
Αρχικά, ορίζουμε την μεταβλητή incomingByte για να αποθηκεύσουμε το byte που θα λαμβάνουμε από την σειριακή επικοινωνία και την μεταβλητή value για να αποθηκεύσουμε την τιμή της μέτρησης από την αναλογική είσοδο A0. Έπειτα, στην setup() το pin 11 ορίζεται ως έξοδος και ξεκινά η σειριακή επικοινωνία στα 9600 baud, ενώ ζητείται από τον χρήστη να εισάγει το γράμμα 'A' ή 'D'. Στην συνέχεια, στην loop() εάν υπάρχουν δεδομένα από τη σειριακή επικοινωνία, ελέγχουμε την τιμή της μεταβλητής incomingByte για να διαπιστώσουμε ποια λειτουργία έχει επιλεγεί:

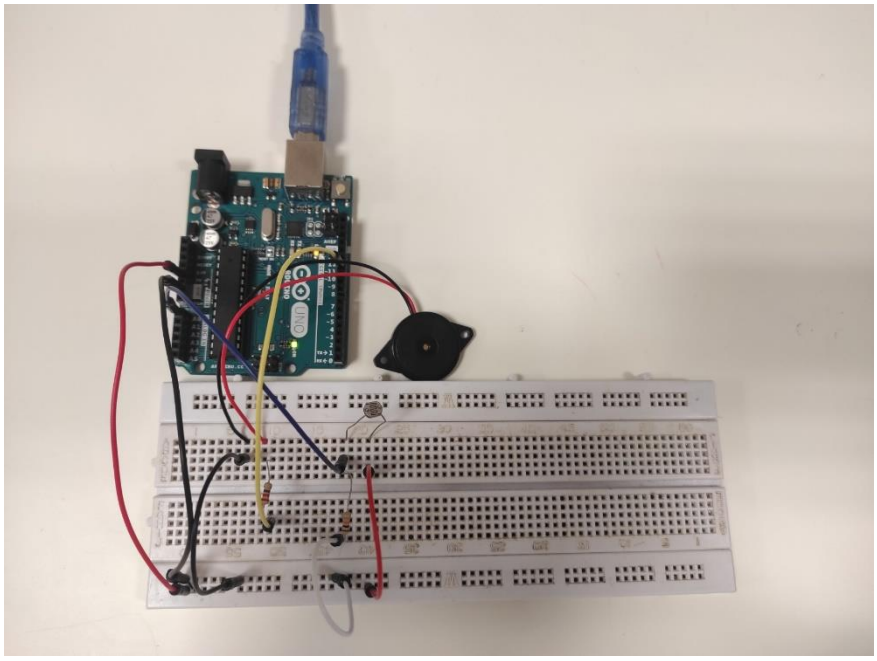
- Εάν η τιμή είναι «A», επιλέγεται η αναλογική λειτουργία και πηγαίνουμε στη συνάρτηση analog(). Εκεί αποθηκεύουμε στη μεταβλητή value την τιμή που διαβάζουμε από την αναλογική είσοδο A0, δηλαδή την τιμή της φωτοαντίστασης, και εμφανίζουμε την αντίστοιχη ένταση στο LED. Στη συνέχεια, αντιστοιχίζουμε την αναλογική τιμή στο εύρος 100 - 1000 Hz (εύρος buzzer) και εκχωρούμε αυτήν τη συχνότητα στο buzzer.
- Εάν η τιμή είναι «D», επιλέγεται η ψηφιακή λειτουργία και πηγαίνουμε στη συνάρτηση digital(). Εκεί αποθηκεύουμε στη μεταβλητή value την τιμή που διαβάζουμε από την αναλογική είσοδο A0 και την εμφανίζουμε. Στη

συνέχεια, αντιστοιχίζουμε την αναλογική τιμή στις ψηφιακές τιμές LOW ή HIGH και εκχωρούμε το αντίστοιχο επίπεδο στο LED.

- Εάν η τιμή είναι οποιαδήποτε άλλη, εμφανίζουμε μήνυμα λάθους.







Παράδειγμα 6: SR04 + DHT11

Ζητούμενα

1. Τροποποιήστε το κύκλωμα και τον κώδικα, ώστε να αναπαράγεται ένας ήχος από ένα buzzer εάν κάτι πλησιάσει σε απόσταση <10 cm.
2. Τι πρόβλημα δημιουργεί η συνάρτηση delay(2000) στο τέλος του κώδικα;
3. Τροποποιήστε τον κώδικα ώστε να αποφευχθεί το παραπάνω πρόβλημα

Κώδικας

```
#include <dht.h>  // Include DHT library for reading temperature and humidity

// Pin definitions
#define trigPin 9        // Trigger pin of ultrasonic sensor
#define echoPin 10       // Echo pin of ultrasonic sensor
#define DHT11_PIN 2      // Data pin for DHT11 sensor
#define BUZZER_PIN 8     // Pin connected to the buzzer

// Variables for sensor measurements
float duration, distance;
float sound_speed;
float temp, hmdt;

// Variables for timing
```

```
unsigned long previousMillis = 0;  // Stores the last time sensor data was
read
const long interval = 2000;         // Interval between readings (2 seconds)

dht DHT;  // Create DHT sensor object

void setup() {
  pinMode(trigPin, OUTPUT);       // Set trigger pin as output
  pinMode(echoPin, INPUT);         // Set echo pin as input
  pinMode(DHT11_PIN, INPUT);       // DHT11 pin as input (even though DHT
library handles it)
  pinMode(BUZZER_PIN, OUTPUT);     // Set buzzer pin as output
  Serial.begin(9600);              // Initialize serial communication at 9600
baud
}

void loop() {
  unsigned long currentMillis = millis();  // Get the current time

  // Only proceed if 2 seconds have passed
  if (currentMillis - previousMillis >= interval) {
    previousMillis = currentMillis;  // Update last reading time

    // Read temperature and humidity from DHT11 sensor
    int chk = DHT.read11(DHT11_PIN);
    temp = DHT.temperature;
    hmdt = DHT.humidity;

    // Trigger the ultrasonic sensor
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    // Read the duration of echo pulse
    duration = pulseIn(echoPin, HIGH);

    // Calculate speed of sound based on temperature and humidity
    sound_speed = 331.4 + (0.606 * temp) + (0.0124 + 0.000058 * hmdt) * temp;

    // Calculate distance based on duration and speed of sound
    distance = (duration / 2) * (sound_speed / 10000);  // Convert to
centimeters

    // Check if distance is within valid range
    if (distance < 400 && distance > 2) {
      Serial.print(distance);
```

```
      Serial.println(" cm");

      // If object is closer than 10 cm, turn on buzzer
      if (distance < 10) {
        digitalWrite(BUZZER_PIN, HIGH);
      } else {
        digitalWrite(BUZZER_PIN, LOW);
      }

    } else {
      Serial.println("Out of range");
      digitalWrite(BUZZER_PIN, LOW);  // Turn off buzzer if out of range
    }

    // Print temperature and humidity
    Serial.print(temp);
    Serial.println(" C");
    Serial.print(hmdt);
    Serial.println(" %");
    Serial.println("===========");
  }
}
```

Επεξήγηση

Αρχικά, ορίζουμε τα pins που θα χρησιμοποιήσουμε για κάθε συσκευή, τις μεταβλητές που θα χρησιμοποιήσουμε για να αποθηκεύσουμε τις μετρήσεις από τους αισθητήρες και δημιουργούμε ένα αντικείμενο της κλάσης dht από την βιβλιοθήκη DHT που επιτρέπει την επικοινωνία με τον αισθητήρα DTH11. Έπειτα, στην setup() αρχικοποιούνται κατάλληλα τα pins και αρχίζει η σειριακή επικοινωνία στα 9600 baud. Στην συνέχεια, στην loop() κάθε 2 δευτερόλεπτα, διαβάζονται η θερμοκρασία και η υγρασία από το DHT11 και εκτελείται μέτρηση απόστασης μέσω του υπερηχητικού αισθητήρα. Η ταχύτητα του ήχου υπολογίζεται δυναμικά, λαμβάνοντας υπόψη τη θερμοκρασία και την υγρασία, για μεγαλύτερη ακρίβεια. Η απόσταση υπολογίζεται από τον χρόνο επιστροφής του ηχητικού παλμού. Αν η απόσταση είναι έγκυρη (μεταξύ 2 και 400 cm), εμφανίζεται στο σειριακό monitor και, αν είναι μικρότερη από 10 cm, ενεργοποιείται το buzzer. Διαφορετικά, το buzzer παραμένει ανενεργό και το πρόγραμμα εμφανίζει την απόσταση, τη θερμοκρασία και την υγρασία στην οθόνη. Αν η μέτρηση είναι εκτός ορίων, τυπώνεται μήνυμα "Out of range" και το buzzer σβήνει.

Η συνάρτηση delay(2000) στο τέλος του κώδικα δημιουργεί το πρόβλημα ότι παγώνει τον μικροελεγκτή για 2 δευτερόλεπτα, καθυστερώντας την εκτέλεση του υπόλοιπου προγράμματος. Κατά τη διάρκεια αυτής της καθυστέρησης, το Arduino δεν μπορεί να ανταποκριθεί σε οποιαδήποτε αλλαγή του περιβάλλοντος ή σε νέα δεδομένα από τους αισθητήρες. Αυτό σημαίνει ότι δεν μπορεί να ανιχνεύσει γρήγορα αλλαγές στην απόσταση, τη θερμοκρασία ή την υγρασία, με αποτέλεσμα να υπάρχει καθυστέρηση στην ενεργοποίηση ή απενεργοποίηση του buzzer, αλλά και γενικότερη καθυστέρηση στις μετρήσεις. Έτσι, η χρήση της delay() περιορίζει την ικανότητα του συστήματος να λειτουργεί σε πραγματικό χρόνο και να αντιδρά άμεσα σε μεταβολές του περιβάλλοντος.

Για να αποφευχθεί το πρόβλημα που δημιουργεί η συνάρτηση delay(2000), μπορούμε να χρησιμοποιήσουμε τη συνάρτηση millis(), η οποία επιτρέπει τον έλεγχο του χρόνου χωρίς να "παγώνει" το πρόγραμμα. Με την millis(), μπορούμε να μετράμε τον χρόνο που έχει περάσει από την τελευταία εκτέλεση μιας ενέργειας (όπως η μέτρηση της απόστασης ή η ανάγνωση των αισθητήρων) και να εκτελούμε τις απαραίτητες ενέργειες χωρίς να καθυστερούμε την υπόλοιπη λειτουργία του προγράμματος. Έτσι, το σύστημα μπορεί να ανταποκρίνεται σε πραγματικό χρόνο, να διαβάζει συνεχώς τα δεδομένα και να αντιδρά άμεσα σε οποιεσδήποτε αλλαγές, χωρίς να "κολλάει" για 2 δευτερόλεπτα σε κάθε κύκλο του loop().

